> 作业2:阅读06-stock-bi-agent代码，回答如下问题:
>
> 1. 什么是前后端分离?
> 2. 历史对话如何存储，以及如何将历史对话作为大模型的下一次输入;

问题1：前后端分离

官方定义：前端项目、后端项目完全独立开发、独立部署、独立运行，只通过接口（API）进行数据通信。

流程：

1. 用户打开浏览器 / APP → **前端页面先加载好**
2. 前端通过 **接口 API** 向后端请求数据
3. 后端只查询数据库、处理逻辑，**只返回纯数据（JSON）**，不返回页面
4. 前端拿到数据，自己渲染到页面上展示

在06-stock-bi-agent项目中，体现在：纯前端（Streamlit） + 纯后端（FastAPI） + 只靠接口通信。

1. 后端提供接口：`/user/login`、`/stock/info`、`/chat/send`
2. 前端**只调用这些接口**，拿到数据后自己渲染页面

前端：

```
./demo/
  - chat/
  - stock/
  - user/
  - mcp/
  - data/
./user/
  - streamlit_demo.py
```

后端

`main_server.py`：后端入口

`/api/`：外部数据接口

`/routers/`：API路由

`/services/`：业务逻辑

`/models/`：数据库

`/agent/`：智能代理

问题2：历史对话如何存储，以及如何将历史对话作为大模型的下一次输入

- 存储：把用户提问 + AI 回答一对一对地存到后端数据库（你的 `conversations.db`）

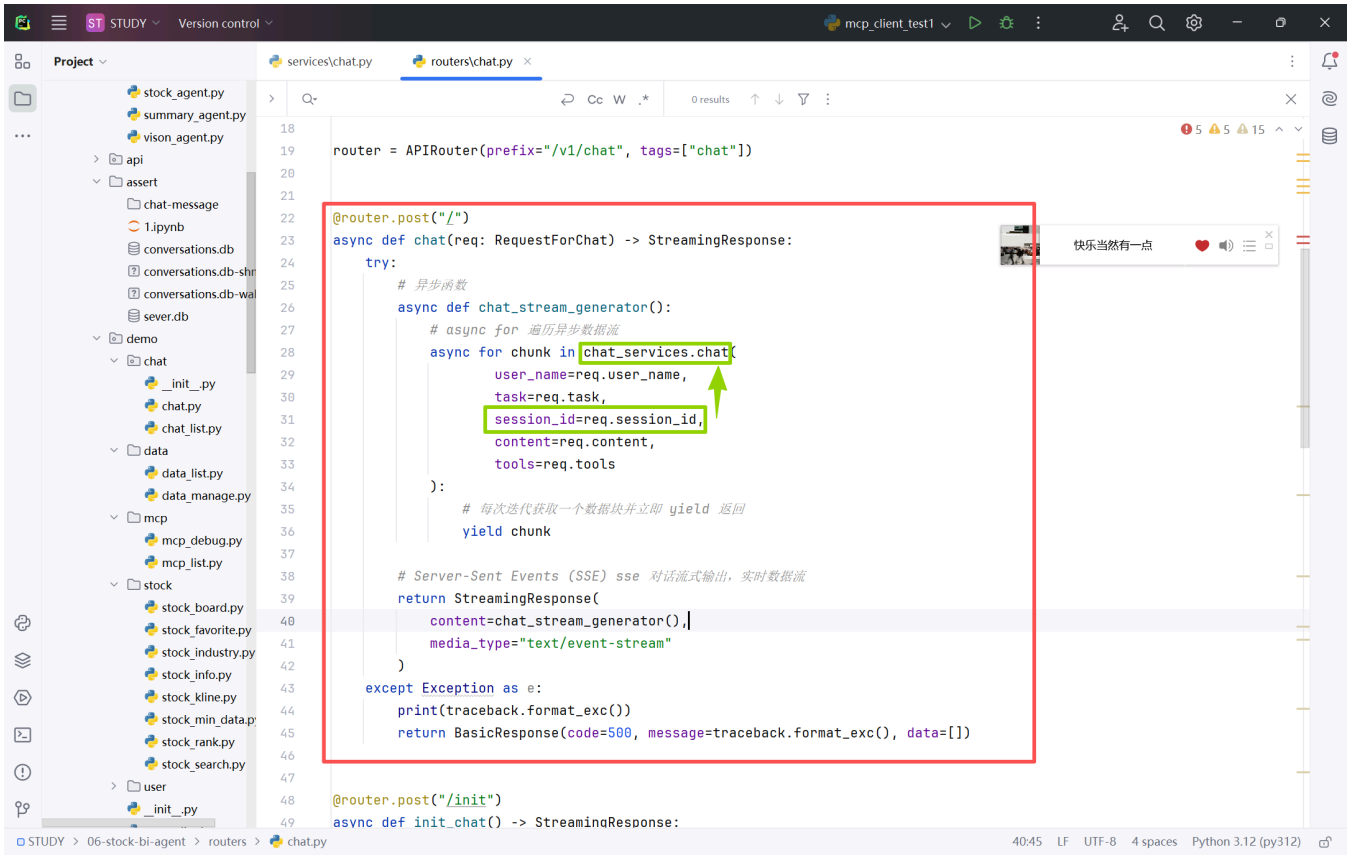
- 传递：每次新提问时，从数据库取出最近 N 条历史，拼进请求里一起发给大模型

1. 前端传来 `session_id`：由 `routers/chat.py` 接收
2. 转发给 `services/chat.py`：真正的历史对话逻辑全部在这里：

存储对话

1. 前端传入对话 ID（路由层：routers/chat.py）

前端通过 `session_id` 标识一段连续对话，路由接口接收并传递给服务层：



> 前端发送聊天消息，就是请求这个接口

存储 1：业务聊天记录（给前端展示）

在 `services/chat.py` 中，使用 `append_message2db` 函数将**用户消息**和**AI 回答**存入数据库 `server.db` 的 `ChatMessageTable` 表：

```python
async def chat(user_name:str, session_id: Optional[str], task: Optional[str], content: str, tools: List[str] = []):
    # 对话管理，通过session id
    if session_id:
        with SessionLocal() as session:
            record = session.query(ChatSessionTable).filter(ChatSessionTable.session_id == session_id).first()
            if not record:
                init_chat_session(user_name, content, session_id, task)

    # 对话记录，存关系型数据库
    append_message2db(session_id, role: "user", content)

    # 获取system message，需要传给大模型，并不能给用户展示
    instructions = get_init_message(task)

    # agent 初始化
    external_client = AsyncOpenAI(
        api_key=os.environ["OPENAI_API_KEY"],
        base_url=os.environ["OPENAI_BASE_URL"],
    )

    # mcp tools 选择
    if not tools or len(tools) == 0:
        tool_mcp_tools_filter: Optional[ToolFilterStatic] = None
    else:
        tool_mcp_tools_filter: ToolFilterStatic = ToolFilterStatic(allowed_tool_names=tools)
    mcp_server = MCPServerSse(
        name="SSE Python Server",
        params={"url": "http://localhost:8900/sse"},
        cache_tools_list=False,
```

存储用户问题

```python
        assistant_message = ""
        async for event in result.stream_events():
            if event.type == "raw_response_event":
                if isinstance(event.data, ResponseTextDeltaEvent): # 如果式大模型的回
                    if event.data.delta:
                        yield f"{event.data.delta}" # sse 不断发给前端
                        assistant_message += event.data.delta

        # 这一条大模型回答，存储对话
        append_message2db(session_id, role: "assistant", assistant_message)

    # 需要调用mcp 服务进行回答
    else:
        async with mcp_server:
            # 哪些工具直接展示结果
            need_viz_tools = ["get_month_line", "get_week_line", "get_day_line", "get_stock_minute_data"]
            if set(need_viz_tools) & set(tools):
                tool_use_behavior = "stop_on_first_tool" # 调用了tool，得到结果，就展示结果
            else:
                tool_use_behavior = "run_llm_again" # 调用了tool，得到结果，继续用大模型的总结结果

            agent = Agent(
                name="Assistant",
                instructions=instructions,
                mcp_servers=[mcp_server],
                model=OpenAIChatCompletionsModel(
                    model=os.environ["OPENAI_MODEL"],
                    openai_client=external_client,
                ),
                tool_use_behavior=tool_use_behavior,
```

存储大模型回答

存储 2：大模型上下文记忆（给 AI 续聊）

在 `services/chat.py` 中，使用 `AdvancedSQLiteSession` 自动管理历史对话，存入框架数据库 `conversations.db`：

```python
async def chat(user_name:str, session_id: Op
    tool_mcp_tools_filter: ToolFilterStatic = ToolFilterStatic(allowed_tool_names=tools)
    mcp_server = MCPServerSse(
        name="SSE Python Server",
        params={"url": "http://localhost:8900/sse"},
        cache_tools_list=False,
        tool_filter=tool_mcp_tools_filter,
        client_session_timeout_seconds=20,
    )

    # openai-agent支持的session存储，存储对话的历史状态
    session = AdvancedSQLiteSession(
        session_id=session_id, # 与 系统中的对话id 关联，存储在关系型数据库中
        db_path="./assert/conversations.db",
        create_tables=True
    )

    # 如果没有选择工具，默认直接调用大模型回答
    if not tools or len(tools) == 0:
        agent = Agent(
            name="Assistant",
            instructions=instructions,
            # mcp_servers=[mcp_server],
            model=OpenAIChatCompletionsModel(
                model=os.environ["OPENAI_MODEL"],
                openai_client=external_client,
            ),
            # tool_use_behavior="stop_on_first_tool",
            model_settings=ModelSettings(parallel_tool_calls=False)
        )

        result = Runner.run_streamed(agent, input=content, session=session) # 流式调用大模型
```

该会话会**自动保存每一轮问答**，无需手动写 SQL。

输入历史对话

在 `services/chat.py` 调用大模型时，**传入携带历史的 `session` 对象**：

```python
        instructions=instructions,
        # mcp_servers=[mcp_server],
        model=OpenAIChatCompletionsModel(
            model=os.environ["OPENAI_MODEL"],
            openai_client=external_client,
        ),
        # tool_use_behavior="stop_on_first_tool",
        model_settings=ModelSettings(parallel_tool_calls=False)
    )

    result = Runner.run_streamed(agent, input=content, session=session) # 流式调用大模型

    assistant_message = ""
    async for event in result.stream_events():
        if event.type == "raw_response_event":
            if isinstance(event.data, ResponseTextDeltaEvent): # 如果式大模型的回答
                if event.data.delta:
                    yield f"{event.data.delta}" # sse 不断发给前端
                    assistant_message += event.data.delta

    # 这一条大模型回答，存储对话
    append_message2db(session_id, role: "assistant", assistant_message)

# 需要调用mcp 服务进行回答
else:
    async with mcp_server:
        # 哪些工具直接展示结果
        need_viz_tools = ["get_month_line", "get_week_line", "get_day_line", "get_stock_minute_data"]
        if set(need_viz_tools) & set(tools):
            tool_use_behavior = "stop_on_first_tool" # 调用了tool，得到结果，就展示结果
```

根据 `session_id` 自动从 `conversations.db` 读取**该对话所有历史**，自动拼接为：系统提示词 + 历史对话 + 新问题，一起发送给大模型。

大模型因此实现**连续对话、上下文记忆。**